R Automation: Functions, mapping, and loops

PSYC 259: Principles of Data Science

John Franchak & Holly O'Rourke

Types of automation

- Replacing manual copy/paste/renaming with data-cleaning scripts
- Replacing drop-down menu analyses with scripts

Types of automation

- Replacing manual copy/paste/renaming with data-cleaning scripts
- Replacing drop-down menu analyses with scripts
- Replacing repetitive code with more efficient code

```
# Tedious thing 1
ds %>% summarize(m_a = mean(a), m_b = mean(b), m_c = mean(c), m_d = mean(d))

# Tedious thing 2
ds_a <- read_csv("data_a.csv")
ds_b <- read_csv("data_b.csv")
ds_c <- read_csv("data_b.csv") #oops
```

```r
# Tedious thing 1
ds %>% summarize(m_a = mean(a), m_b = mean(b), m_c = mean(c), m_d = mean(d))
ds %>% summarize(across(a:d), mean)

# Tedious thing 2
ds_a <- read_csv(“data_a.csv”)
ds_b <- read_csv(“data_b.csv”)
ds_c <- read_csv(“data_b.csv”) #oops
ds <- read_csv(str_glue("data_{c('a','b','c')}.csv"))
```

Today's tutorial

- Ways to make coding more efficient
  - Understanding and creating functions
  - Mutating and summarizing *across* columns
  - Repeating operations using for loops
  - Repeating operations using *map*

Follow along from the Github repo

Last updated: 2026-10-23

Part 1: Functions

What is a function?

- Anything in R that you call with `function_name()`
- A function does three main things:

1. Takes input arguments (or reads input data from the current environment)
2. Runs code on those arguments/data
3. Returns some output

Defining a simple function

- The keyword `function()` tells R to create a function
- Functions have to be assigned to a name (just like anything else)
- Input arguments go inside the parentheses
- Everything between the curly braces runs when you call the function
- `return()` tells the function what to output

```
say_hello <- function(input_name) {
  hello_output <- str_glue("Hello, {input_name}!")
  return(hello_output)
}
say_hello(input_name = "class")
```

```
Hello, class!
```

```
hello_world <- say_hello("world!")
print(hello_world)
```

```
Hello, world!!
```

Functions extend and customize a programming language

- R packages aren't anything more than sets of functions that you load using `library()`
- CRAN is a centralized repository of vetted R packages for thousands of different purposes
  - `install.packages()` installs packages that are hosted on CRAN
- You can also download experimental packages or newer versions from Github
  - `devtools` lets you install directly from Github links
- You can also write your own functions that you load for a project to make repetitive operations easier

Defining a function with multiple arguments

- Named input *arguments* go in parentheses
- The function returns the last computed value or the value listed by the `return()` function
- Be sure to highlight and run the entire function to assign it

```
hyp <- function(side_a, side_b) {
  a <- side_a^2
  b <- side_b^2
  h <- sqrt(a + b)
  return(h)
}
hyp(3, 4)
```

```
[1] 5
```

```
hyp(9, 12)
```

Function arguments

- Functions can have any number of arguments
- To make arguments optional, give a default value
- `NULL` can be a useful default option

```
hyp <- function(side_a, side_b = NULL) {
  a <- side_a^2
  b <- ifelse(is.null(side_b), a, side_b^2)
  h <- sqrt(a + b)
  return(h)
}
hyp(9, 9)
```

```
[1] 12.72792
```

```
hyp(9)
```

```
[1] 12.72792
```

Function arguments

```
hyp <- function(side_a, side_b) {
  #Functions don't ensure the type of arguments
  if (is.numeric(side_a) & is.numeric(side_b)) {
      a <- side_a^2
      b <- ifelse(is.null(side_b), a, side_b^2)
      h <- sqrt(a + b)
      return(h)
  } else {
    return("Both sides must be numeric values")
  }
}
hyp(9, 9)
```

```
[1] 12.72792
```

```
hyp("9", "8")
```

```
[1] "Both sides must be numeric values"
```

Control program flow with `if` and `else` statements

```
x <- 1
if (x == 0) {
  paste(x, "is exactly 0")
} else if (x < 0) {
  paste(x, "is negative")
} else {
  paste(x, "is positive")
}
```

```
[1] "1 is positive"
```

Control program flow with `if` and `else` statements

```
x <- -1
if (x == 0) {
  paste(x, "is exactly 0")
} else if (x < 0) {
  paste(x, "is negative")
} else {
  paste(x, "is positive")
}
```

```
[1] "-1 is negative"
```

Write "wrapper functions" to make your life easier

```
mean_na <- function(x) mean(x, na.rm = T)
sd_na <- function(x) sd(x, na.rm = T)
se_na <- function(x) {
  x <- discard(x, is.na) #discard is like "filter if not" for vectors
  sd(x)/sqrt(length(x))
}
```

- Note, one-line functions don't need curly brackets

Use lists to return multiple outputs

```
summary_stats_na <- function(x) {
  # Functions can call previously defined functions
  list(mean = mean_na(x), sd = sd_na(x), se = se_na(x))
}
data <- c(4, 6, 9, 12, 5, 6, NA)
summary_stats_na(data)
```

```
$mean
[1] 7

$sd
[1] 2.966479

$se
[1] 1.21106
```

Piped expressions can be saved as functions

```
avg_height_mass <- . %>%
  select(height, mass) %>%
  summarize(mean_ht = mean_na(height), mean_ms = mean_na(mass))

starwars %>% avg_height_mass
```

```
# A tibble: 1 × 2
  mean_ht mean_ms
    <dbl>   <dbl>
1    175.    97.3
```

```
starwars %>% group_by(species) %>% avg_height_mass
```

```
# A tibble: 38 × 3
   species   mean_ht mean_ms
   <chr>       <dbl>   <dbl>
 1 Aleena         79     15
 2 Besalisk      198    102
 3 Cerean        198     82
 4 Chagrian      196    NaN
```

Advantages of functions

- Save typing, make code more readable
- Encapsulate computations
    - Note that *internal* intermediate variables don't appear in your environment
    - Functions in R can access variables in the global environment even if they are not passed as arguments
- Introduce abstraction
    - Breaks up your code into conceptual chunks
    - Increases the possibility of reuse (and spend more time debugging things that will be reused)

Part 2: Automation with across

Some types of automation are built-in

- *Vectorized* operations (like adding values between two columns) operate on every element simultaneously (you don't have to perform the operation on every row)
- `rename_all()`, `rename_with()` and `clean_names()` operate on multiple columns
- `mutate()` and `summarize()` can operate on selections of columns with `across()`
  - `across(selection_statement, fx_to_apply)`

Mutate across()

```
ds <- starwars %>% select(-where(is.list))
ds
```

```
# A tibble: 87 × 11
   name     height  mass hair_color skin_color eye_color birth_year sex   gender
   <chr>     <int> <dbl> <chr>      <chr>      <chr>           <dbl> <chr> <chr>
 1 Luke Sk…    172    77 blond      fair       blue             19   male  mascu…
 2 C-3PO       167    75 <NA>       gold       yellow          112   none  mascu…
 3 R2-D2        96    32 <NA>       white, bl… red              33   none  mascu…
 4 Darth V…    202   136 none       white      yellow           41.9 male  mascu…
 5 Leia Or…    150    49 brown      light      brown            19   fema… femin…
 6 Owen La…    178   120 brown, gr… light      blue             52   male  mascu…
 7 Beru Wh…    165    75 brown      light      blue             47   fema… femin…
 8 R5-D4        97    32 <NA>       white, red red              NA   none  mascu…
 9 Biggs D…    183    84 black      light      brown            24   male  mascu…
10 Obi-Wan…    182    77 auburn, w… fair       blue-gray        57   male  mascu…
# ℹ 77 more rows
# ℹ 2 more variables: homeworld <chr>, species <chr>
```

Mutate across()

```
ds <- starwars %>% select(-where(is.list))
ds %>% mutate(across(everything(), as.character))
```

```
# A tibble: 87 × 11
   name     height mass  hair_color skin_color eye_color birth_year sex   gender
   <chr>    <chr>  <chr> <chr>      <chr>      <chr>     <chr>      <chr> <chr>
 1 Luke Sk… 172    77    blond      fair       blue      19         male  mascu…
 2 C-3PO    167    75    <NA>       gold       yellow    112        none  mascu…
 3 R2-D2    96     32    <NA>       white, bl… red       33         none  mascu…
 4 Darth V… 202    136   none       white      yellow    41.9       male  mascu…
 5 Leia Or… 150    49    brown      light      brown     19         fema… femin…
 6 Owen La… 178    120   brown, gr… light      blue      52         male  mascu…
 7 Beru Wh… 165    75    brown      light      blue      47         fema… femin…
 8 R5-D4    97     32    <NA>       white, red red       <NA>       none  mascu…
 9 Biggs D… 183    84    black      light      brown     24         male  mascu…
10 Obi-Wan… 182    77    auburn, w… fair       blue-gray 57         male  mascu…
# ℹ 77 more rows
# ℹ 2 more variables: homeworld <chr>, species <chr>
```

Mutate across()

```
ds <- starwars %>% select(-where(is.list))
ds %>% mutate(across(c("height", "mass"), round))
```

```
# A tibble: 87 × 11
   name     height  mass hair_color skin_color eye_color birth_year sex   gender
   <chr>     <dbl> <dbl> <chr>      <chr>      <chr>           <dbl> <chr> <chr>
 1 Luke Sk…    172    77 blond      fair       blue             19   male  mascu…
 2 C-3PO       167    75 <NA>       gold       yellow          112   none  mascu…
 3 R2-D2        96    32 <NA>       white, bl… red              33   none  mascu…
 4 Darth V…    202   136 none       white      yellow           41.9 male  mascu…
 5 Leia Or…    150    49 brown      light      brown            19   fema… femin…
 6 Owen La…    178   120 brown, gr… light      blue             52   male  mascu…
 7 Beru Wh…    165    75 brown      light      blue             47   fema… femin…
 8 R5-D4        97    32 <NA>       white, red red              NA   none  mascu…
 9 Biggs D…    183    84 black      light      brown            24   male  mascu…
10 Obi-Wan…    182    77 auburn, w… fair       blue-gray        57   male  mascu…
# ℹ 77 more rows
# ℹ 2 more variables: homeworld <chr>, species <chr>
```

Mutate across()

```
ds <- starwars %>% select(-where(is.list))
ds %>% mutate(across(where(is.numeric), round, .names = "{.col}_rounded"), .after = name)
```

```
# A tibble: 87 × 14
   name   height_rounded mass_rounded birth_year_rounded height  mass hair_color
   <chr>           <dbl>        <dbl>               <dbl>  <int> <dbl> <chr>
 1 Luke …            172           77                  19    172    77 blond
 2 C-3PO             167           75                 112    167    75 <NA>
 3 R2-D2              96           32                  33     96    32 <NA>
 4 Darth…            202          136                  42    202   136 none
 5 Leia …            150           49                  19    150    49 brown
 6 Owen …            178          120                  52    178   120 brown, gr…
 7 Beru …            165           75                  47    165    75 brown
 8 R5-D4              97           32                  NA     97    32 <NA>
 9 Biggs…            183           84                  24    183    84 black
10 Obi-W…            182           77                  57    182    77 auburn, w…
# ℹ 77 more rows
# ℹ 7 more variables: skin_color <chr>, eye_color <chr>, birth_year <dbl>,
#   sex <chr>, gender <chr>, homeworld <chr>, species <chr>
```

Mutate across()

```
ds <- starwars %>% select(-where(is.list))
ds %>% mutate(across(where(is.numeric), ~ .x *100))
```

```
# A tibble: 87 × 11
   name     height  mass hair_color skin_color eye_color birth_year sex   gender
   <chr>     <dbl> <dbl> <chr>      <chr>      <chr>           <dbl> <chr> <chr>
 1 Luke Sk…  17200  7700 blond      fair       blue             1900 male  mascu…
 2 C-3PO     16700  7500 <NA>       gold       yellow          11200 none  mascu…
 3 R2-D2      9600  3200 <NA>       white, bl… red              3300 none  mascu…
 4 Darth V…  20200 13600 none       white      yellow           4190 male  mascu…
 5 Leia Or…  15000  4900 brown      light      brown            1900 fema… femin…
 6 Owen La…  17800 12000 brown, gr… light      blue             5200 male  mascu…
 7 Beru Wh…  16500  7500 brown      light      blue             4700 fema… femin…
 8 R5-D4      9700  3200 <NA>       white, red red                NA none  mascu…
 9 Biggs D…  18300  8400 black      light      brown            2400 male  mascu…
10 Obi-Wan…  18200  7700 auburn, w… fair       blue-gray        5700 male  mascu…
# ℹ 77 more rows
# ℹ 2 more variables: homeworld <chr>, species <chr>
```

Mutate across()

```
ds <- starwars %>% select(-where(is.list))
ds %>% mutate(across(ends_with("color"), ~ str_remove_all(.x, "[:punct:]")))
```

```
# A tibble: 87 × 11
   name     height  mass hair_color skin_color eye_color birth_year sex   gender
   <chr>     <int> <dbl> <chr>      <chr>      <chr>          <dbl> <chr> <chr>
 1 Luke Sk…    172    77 blond      fair       blue            19   male  mascu…
 2 C-3PO       167    75 <NA>       gold       yellow         112   none  mascu…
 3 R2-D2        96    32 <NA>       white blue red             33   none  mascu…
 4 Darth V…    202   136 none       white      yellow          41.9 male  mascu…
 5 Leia Or…    150    49 brown      light      brown           19   fema… femin…
 6 Owen La…    178   120 brown grey light      blue            52   male  mascu…
 7 Beru Wh…    165    75 brown      light      blue            47   fema… femin…
 8 R5-D4        97    32 <NA>       white red  red             NA   none  mascu…
 9 Biggs D…    183    84 black      light      brown           24   male  mascu…
10 Obi-Wan…    182    77 auburn wh… fair       bluegray        57   male  mascu…
# ℹ 77 more rows
# ℹ 2 more variables: homeworld <chr>, species <chr>
```

Across() relies on "in-line" functions

- `~ round(.x)` is a shortcut for:

```
round_x <- function(x) round(x)
```

- in-line functions use `.x` as the argument

Summarize across()

```
ds <- starwars %>% select(-where(is.list))
fx_list <- list(mean = mean_na, sd = sd_na, se = se_na)
ds %>% summarize(across(height:mass, ~ mean(.x, na.rm = T)))
```

```
# A tibble: 1 × 2
  height  mass
   <dbl> <dbl>
1   175.  97.3
```

Summarize across()

```
ds <- starwars %>% select(-where(is.list))
fx_list <- list(mean = mean_na, sd = sd_na, se = se_na)
ds %>% summarize(across(height:mass, mean_na))
```

```
# A tibble: 1 × 2
  height  mass
   <dbl> <dbl>
1   175.  97.3
```

Summarize across()

```
ds <- starwars %>% select(-where(is.list))
fx_list <- list(mean = mean_na, sd = sd_na, se = se_na)
ds %>% summarize(across(where(is.numeric), fx_list))
```

```
# A tibble: 1 × 9
  height_mean height_sd height_se mass_mean mass_sd mass_se birth_year_mean
        <dbl>     <dbl>     <dbl>     <dbl>   <dbl>   <dbl>           <dbl>
1        175.      34.8      3.86      97.3    169.    22.1            87.6
# ℹ 2 more variables: birth_year_sd <dbl>, birth_year_se <dbl>
```

Summarize across()

```
ds <- starwars %>% select(-where(is.list))
fx_list <- list(mean = mean_na, sd = sd_na, se = se_na)
ds %>% summarize(across(where(is.numeric), fx_list, .names = "{.fn}_{.col}"))
```

```
# A tibble: 1 × 9
  mean_height sd_height se_height mean_mass sd_mass se_mass mean_birth_year
        <dbl>     <dbl>     <dbl>     <dbl>   <dbl>   <dbl>           <dbl>
1        175.      34.8      3.86      97.3    169.    22.1            87.6
# ℹ 2 more variables: sd_birth_year <dbl>, se_birth_year <dbl>
```

across() uses `str_glue` syntax helpers

```
ds %>% summarize(across(where(is.numeric), fx_list, .names = "{.fn}_{.col}"))
```

- glue syntax means replace what is in curly brackets with a value
- `.fn` represents name of the function being applied (mean, sd)
- `.col` represents name of the column being summarized/mutated (height, mass)
- Returns nicely named columns (e.g., mean_height, sd_mass)

Make a summary stats function

```
summary_stats <- function(df, var_select_string) {
  na_mean <- function(x) mean(x, na.rm = T)
  na_sd <- function(x) sd(x, na.rm = T)
  fx_list <- list(M = na_mean, SD = na_sd)
  df  %>% summarize(across(contains(var_select_string) & where(is.numeric), fx_list))
}
ds %>% summary_stats("height")
```

```
# A tibble: 1 × 2
  height_M height_SD
     <dbl>     <dbl>
1     175.      34.8
```

```
ds %>% group_by(species) %>% filter(n() > 1) %>% summary_stats(c("height","mass"))
```

```
# A tibble: 9 × 5
  species  height_M height_SD mass_M mass_SD
  <chr>       <dbl>     <dbl>  <dbl>   <dbl>
1 Droid        131.     49.1    69.8   51.0
2 Gungan       209.     14.2    74     11.3
3 Human        178      12.0    81.3   19.3
4 Kaminoan     221      11.3    88     NA
5 Mirialan     168       2.83   53.1    4.28
```

Part 3: Iteration with for loops and map

Iteration

- Run the same series of computations on each element of a vector or list, one at a time
- A "for loop" is constructed in a similar way to a function or if-else statement

```
for (i in c(1, 2, 3, 4, 5)) {
  # "i" will take on each value of the vector 1:5
  print(paste("This is loop iteration", i))
}
```

```
[1] "This is loop iteration 1"
[1] "This is loop iteration 2"
[1] "This is loop iteration 3"
[1] "This is loop iteration 4"
[1] "This is loop iteration 5"
```

For loops

- The vector to iterate over doesn't need to be numeric or sequential:

```
days_of_the_week <- c("Tues", "Wed", "Mon", "Thur", "Sun", "Fri" )

for (i in days_of_the_week) {
  print(paste("Today is", i))
}
```

```
[1] "Today is Tues"
[1] "Today is Wed"
[1] "Today is Mon"
[1] "Today is Thur"
[1] "Today is Sun"
[1] "Today is Fri"
```

Use for loops to run a series of analyses

```
mtcars #built-in R dataset
```

```
                   mpg cyl  disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
Duster 360        14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
Merc 240D         24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
Merc 230          22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
Merc 280          19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
Merc 280C         17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
Merc 450SE        16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
Merc 450SL        17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
Merc 450SLC       15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
Cadillac Fleetwood  10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
Chrysler Imperial   14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
Fiat 128          32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
Honda Civic       30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
Toyota Corolla    33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
Toyota Corona     21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
```

Use for loops to run a series of analyses

```
predictors <- c("cyl", "disp", "hp", "drat", "wt", "qsec")
cor_output <- vector() #empty vector to store results
for (p in predictors) {
  #Calculate each predictor's correlation with 'mpg'
  cor_output[p] <- cor(mtcars['mpg'], mtcars[p])
}
cor_output
```

```
       cyl       disp         hp       drat         wt       qsec 
-0.8521620 -0.8475514 -0.7761684  0.6811719 -0.8676594  0.4186840 
```

Use for loops to run a series of analyses

```
# Simple linear regression
lm(mpg ~ cyl, data = mtcars)
```

```
Call:
lm(formula = mpg ~ cyl, data = mtcars)

Coefficients:
(Intercept)          cyl
     37.885       -2.876
```

```
lm(mpg ~ disp, data = mtcars)
```

```
Call:
lm(formula = mpg ~ disp, data = mtcars)

Coefficients:
(Intercept)         disp
   29.59985     -0.04122
```

```
lm(mpg ~ hp, data = mtcars)
```

Use for loops to run a series of analyses

```
predictors <- c("cyl", "disp", "hp", "drat", "wt", "qsec")
for (p in predictors) {
  f <- as.formula(str_glue("mpg ~ {p}")) # Convert a string to a formula
  res <- lm(f, data = mtcars)
  print(p) #Need to explicitly print things within loops so that they display
  print(summary(res))
}
```

```
[1] "cyl"

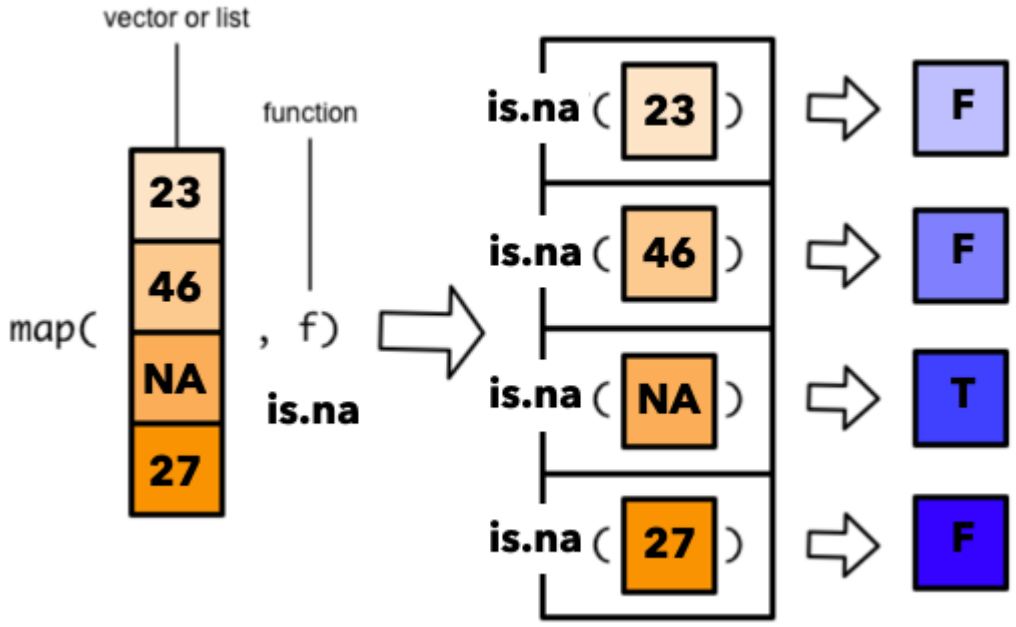
Call:
lm(formula = f, data = mtcars)

Residuals:
    Min      1Q  Median      3Q     Max 
-4.9814 -2.1185  0.2217  1.0717  7.5186 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  37.8846     2.0738   18.27  < 2e-16 ***
cyl          -2.8758     0.3224   -8.92 6.11e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 3.206 on 30 degrees of freedom
```

Map a function to each element of a vector/list



Map a function to each element of a vector/list

```
days_of_the_week <- c("Mon", "Tues", "Wed", "Thur", "Fri", "Sat", "Sun")
map(days_of_the_week, toupper) #Default output is a list
```

```
[[1]]
[1] "MON"

[[2]]
[1] "TUES"

[[3]]
[1] "WED"

[[4]]
[1] "THUR"

[[5]]
[1] "FRI"

[[6]]
[1] "SAT"

[[7]]
```

Map a function to each element of a vector/list

```
days_of_the_week <- c("Mon", "Tues", "Wed", "Thur", "Fri", "Sat", "Sun")
map_chr(days_of_the_week, toupper) #Use map_* to specify the output type
```

```
[1] "MON"  "TUES" "WED"  "THUR" "FRI"  "SAT"  "SUN"
```

```
map_chr(days_of_the_week, ~ str_glue("Day: {.x}")) #Use in-line fns as in across()
```

```
[1] "Day: Mon"  "Day: Tues" "Day: Wed"  "Day: Thur" "Day: Fri"  "Day: Sat"
[7] "Day: Sun"
```

```
map_lgl(days_of_the_week, ~ str_detect(.x, "M"))
```

```
[1]  TRUE FALSE FALSE FALSE FALSE FALSE FALSE
```

Use map to run a series of analyses

```
predictors <- c("cyl", "disp", "hp", "drat", "wt", "qsec")
cor_output <- map(predictors, ~ cor(mtcars['mpg'], mtcars[.x]))
cor_output
```

```
[[1]]
          cyl
mpg -0.852162

[[2]]
          disp
mpg -0.8475514

[[3]]
            hp
mpg -0.7761684

[[4]]
         drat
mpg 0.6811719

[[5]]
            wt
mpg -0.8676594

[[6]]
```

Use map to run a series of analyses

```
predictors <- c("cyl", "disp", "hp", "drat", "wt", "qsec")
cor_output <- map_dbl(predictors, ~ cor(mtcars['mpg'], mtcars[.x]))
cor_output
```

```
[1] -0.8521620 -0.8475514 -0.7761684  0.6811719 -0.8676594  0.4186840
```

```
# Use set_names to make mapped outputs more readable
cor_output %>% set_names(predictors)
```

```
       cyl       disp         hp       drat         wt       qsec
-0.8521620 -0.8475514 -0.7761684  0.6811719 -0.8676594  0.4186840
```

Use map to run a series of analyses

```
predictors <- c("cyl", "disp", "hp", "drat", "wt", "qsec")
output <- map(predictors, ~ lm(as.formula(str_glue("mpg ~ {.x}")), data = mtcars))
output %>% set_names(predictors)
```

```
$cyl

Call:
lm(formula = as.formula(str_glue("mpg ~ {.x}")), data = mtcars)

Coefficients:
(Intercept)          cyl
     37.885       -2.876


$disp

Call:
lm(formula = as.formula(str_glue("mpg ~ {.x}")), data = mtcars)

Coefficients:
(Intercept)         disp
   29.59985     -0.04122


$hp
```

Other considerations

- If you're new to loops/mapping, don't try to write the whole thing at once
- First write the single element/single case computation, and then edit it to work over a loop/map
- Writing a function makes map or across a lot easier to use (no need for in-line functions)
- Splitting is a nice way to create a list of filtered data frames to feed into map

Use map to run a series of analyses

```
predictors <- c("cyl", "disp", "hp", "drat", "wt", "qsec")
mpg_lm <- function(p) {
  f <- as.formula(str_glue("mpg ~ {p}"))
  res <- lm(f, data = mtcars)
  res$call <- f
  return(res)
}
results <- map(predictors, mpg_lm) # returns list of model results
map(results, summary) # runs summary on each item in the list
```

```
[[1]]

Call:
mpg ~ cyl

Residuals:
    Min      1Q  Median      3Q     Max
-4.9814 -2.1185  0.2217  1.0717  7.5186

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  37.8846     2.0738   18.27  < 2e-16 ***
cyl          -2.8758     0.3224   -8.92 6.11e-10 ***
---
```

What if we wanted all combinations?

```
preds <- c("cyl", "disp", "hp", "drat", "wt", "qsec")
all_combos <- expand.grid(preds, preds) %>% filter(Var1 != Var2)
all_combos
```

```
   Var1 Var2
1  disp  cyl
2    hp  cyl
3  drat  cyl
4    wt  cyl
5  qsec  cyl
6   cyl disp
7    hp disp
8  drat disp
9    wt disp
10 qsec disp
11  cyl   hp
12 disp   hp
13 drat   hp
14   wt   hp
15 qsec   hp
16  cyl drat
17 disp drat
18   hp drat
19   wt drat
```

Use map to run a series of analyses

```r
# Revise fx to take two inputs
mpg_lm <- function(criterion, predictor) {
  f <- as.formula(str_glue("{criterion} ~ {predictor}"))
  res <- lm(f, data = mtcars)
  res$call <- f
  return(res)
}
# Use map2 to provide two inputs
results <- map2(all_combos$Var1, all_combos$Var2, mpg_lm)
map(results, summary)
```

```
[[1]]

Call:
disp ~ cyl

Residuals:
    Min      1Q  Median      3Q     Max
-73.985 -45.233   3.565  26.688 127.818

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) -156.609     35.181  -4.452 0.000109 ***
cyl           62.599      5.469  11.445  1.8e-12 ***
```

Use the broom package to tidy results

```
library(broom) #for model output 'tidy' function
mpg_lm <- function(criterion, predictor) {
  f <- as.formula(str_glue("{criterion} ~ {predictor}"))
  res <- lm(f, data = mtcars)
  res <- tidy(res) %>% mutate(criterion = criterion, predictor = predictor, .before = 1)
  return(res)
}
# Use map2_dfr to provide two inputs and combine results into a dataframe
map2_dfr(all_combos$Var1, all_combos$Var2, mpg_lm)
```

```
# A tibble: 60 × 7
   criterion predictor term        estimate std.error statistic  p.value
   <fct>     <fct>     <chr>          <dbl>     <dbl>     <dbl>    <dbl>
 1 disp      cyl       (Intercept) -157.      35.2        -4.45 1.09e- 4
 2 disp      cyl       cyl           62.6      5.47       11.4  1.80e-12
 3 hp        cyl       (Intercept)  -51.1     25.0        -2.04 4.99e- 2
 4 hp        cyl       cyl           32.0      3.88        8.23 3.48e- 9
 5 drat      cyl       (Intercept)    4.89     0.251      19.5  1.40e-18
 6 drat      cyl       cyl           -0.210    0.0390     -5.37 8.24e- 6
 7 wt        cyl       (Intercept)    0.565    0.401       1.41 1.69e- 1
 8 wt        cyl       cyl            0.429    0.0623      6.88 1.22e- 7
 9 qsec      cyl       (Intercept)   21.5      0.948      22.7  1.91e-20
10 qsec      cyl       cyl           -0.592    0.147      -4.02 3.66e- 4
# ℹ 50 more rows
```

Split data to apply function to each subset

```
ds <- starwars %>% mutate(species = factor(
    ifelse(species %in% c("Human", "Droid"), species, "Other")))

height_mass_lm <- function(df) {
  df %>% drop_na(height, mass) %>%
    lm(mass ~ height, data = .) %>%
    tidy %>% mutate(species = df$species[1], .before = 1)}

split_ds <- split(ds, ds$species) #base R fx
map_dfr(split_ds, height_mass_lm)
```

```
# A tibble: 6 × 6
  species term        estimate std.error statistic p.value
  <fct>   <chr>          <dbl>     <dbl>     <dbl>   <dbl>
1 Droid   (Intercept)  -62.1      28.7      -2.16    0.163
2 Droid   height         0.942     0.195     4.83    0.0403
3 Human   (Intercept)  -81.8      60.6      -1.35    0.194
4 Human   height         0.905     0.336     2.70    0.0148
5 Other   (Intercept)    6.68    166.        0.0402  0.968
6 Other   height         0.588     0.926     0.636   0.529
```

Split data and write subset to file

```
ds <- starwars %>% mutate(species = factor(
    ifelse(species %in% c("Human", "Droid"), species, "Other")))

write_species <- function(df) {
  fname <- str_glue("data_species/{df$species[1]}.csv")
  df %>% write_csv(file = fname) }

split_ds <- split(ds, ds$species)
walk(split_ds, write_species) #Walk is map w/o returning data
```